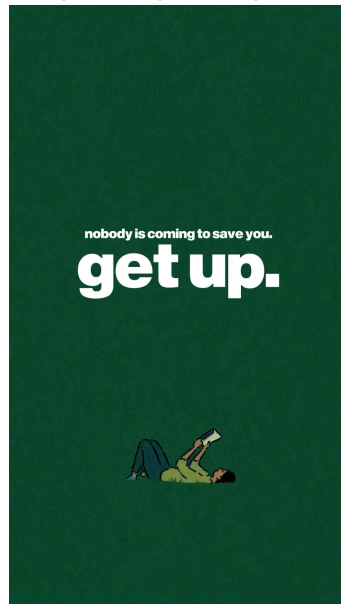


Lab on Recursion



Good

Morning

Content

- Power function
- Indices of Array
- Tower of Hanoi

* Power function

Given two integers a & n . Find a^n using recursion

$$\left. \begin{array}{l} a=2 \\ n=3 \end{array} \right\} 2^3 = 2 * 2 * 2 = 8$$

$$\left. \begin{array}{l} a=4 \\ n=2 \end{array} \right\} 4^2 = 4 * 4 = 16$$

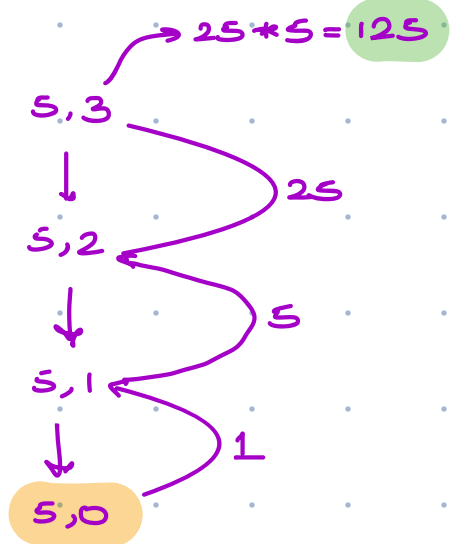
$$a^n = \underbrace{a * a * a * \dots * a * a}_{n \text{ times}}$$

$$a^{n-1} = \underbrace{a * a * a * \dots * a}_{n-1 \text{ times}}$$

$$a^n = a^{n-1} * a \longrightarrow \text{pow}(a, n) = \text{pow}(a, n-1) * a$$

```
long pow(a, n) {  
    if (n == 0) return 1;  
    int rec = pow(a, n-1);  
    return a * rec;  
}
```

TC: $O(N)$
SC: $O(N)$



02. Idea 2

$$2^{10} \rightarrow 2^5 * 2^5$$

$$2^{11} \rightarrow 2^5 * 2^5 * 2$$

$$3^{12} \rightarrow 3^6 * 3^6$$

$$3^{13} \rightarrow 3^6 * 3^6 * 3$$

a^n : n is even

$$a^n \rightarrow a^{n/2} * a^{n/2}$$

a^n : n is odd

$$a^n \rightarrow a^{n/2} * a^{n/2} * a$$

```
long pow(a, n)
```

```
    if (n == 0) return 1
```

```
    if (n % 2 == 0)
```

```
        return pow(a, n/2) * pow(a, n/2);
```

```
    else {
```

```
        return pow(a, n/2) * pow(a, n/2) * a;
```

```
    }
```

5, 11



5, 5



5, 2



5, 1



5, 0

Total time taken by $\text{pow}(a, n) = f(n)$

Total time taken by $\text{pow}(a, n/2) = f(n/2)$

$$f(n) = f(n/2) + f(n/2) + 1$$

$$f(n) = 2f(n/2) + 1$$

$$f(0) = 1$$

$$f(1) = 1$$

Recurrence relation

$$f(n) = 2f(n/2) + 1 = 2^1 f(n/2^1) + 2^1 - 1 \rightarrow 1$$

$$f(n/2) = 2f(n/4) + 1$$

$$f(n) = 2 * (2f(n/4) + 1) + 1$$

$$f(n) = 4f(n/4) + 3 = 2^2 f(n/2^2) + 2^2 - 1 \rightarrow 2$$

$$f(n/4) = 2f(n/8) + 1$$

$$\begin{aligned} f(n) &= 4 * (2f(n/8) + 1) + 3 \\ &= 8f(n/8) + 7 = 2^3 f(n/2^3) + 2^3 - 1 \rightarrow 3 \end{aligned}$$

After k substitution

$$f(n) = 2^k f(n/2^k) + 2^k - 1$$

$$f(0) = 1 \quad \times$$

$$f(1) = 1 \quad \checkmark$$

$$\frac{N}{2^k} = 0$$

$$N = 0 \quad \times$$

$$\frac{N}{2^k} = 1$$

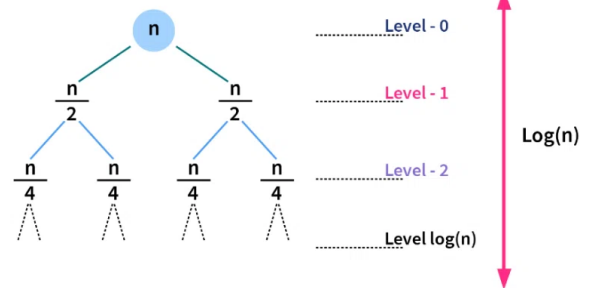
$$N = 2^k$$

$$\rightarrow k = \log N$$

$$\begin{aligned} f(n) &= N * 1 + N - 1 \\ &= 2N - 1 \end{aligned}$$

$$TC \approx O(N)$$

$$SC = O(\log N)$$



```
long pow(a, n)
```

```
    if (n == 0) return 1
```

```
    int rec = pow(a, n/2)
```

```
    if (n % 2 == 0) return rec * rec;
```

```
    else return rec * rec * a;
```

```
    }
```

$$TC: O(\log N)$$

$$SC: O(\log N)$$

Ans = $25 * 25 * 5$

long pow(a, n) 5, 5

if (n == 0) return 1

int rec = pow(a, n/2) = 25

if (n % 2 == 0) return rec * rec;

else return rec * rec * a;

long pow(a, n) 5, 2

if (n == 0) return 1

int rec = pow(a, n/2) = 5

if (n % 2 == 0) return rec * rec;

else return rec * rec * a;

long pow(a, n) 5, 1

if (n == 0) return 1

int rec = pow(a, n/2) = 1

if (n % 2 == 0) return rec * rec;

else return rec * rec * a;

long pow(a, n) 5, 0

if (n == 0) return 1

int rec = pow(a, n/2)

if (n % 2 == 0) return rec * rec;

else return rec * rec * a;

7:58 → 8:10 AM

* All indices of Array

Problem Statement

Given an array of integers A with N elements and a target integer B, the task is to find all the indices at which B occurs in the array.

It is guaranteed that the target B, exist atleast once in the Array A.

$$A[] = \{ \underset{0}{4}, \underset{1}{5}, \underset{2}{3}, \underset{3}{1}, \underset{4}{5}, \underset{5}{4}, \underset{6}{5} \} \quad B = 5$$

$$Ans[] = [\underset{0}{1}, \underset{1}{4}, \underset{2}{6}]$$

Find all the Indices at which B occurs in the given array, A:

A = [1, 2, 3, 1, 1]

B = 1

$$Ans[] = [0, 3, 4]$$

Observation → Need two variables

Iterate on array

count no. of ele equals to target

recur([] A, B, idx, cnt)

$$A[] = \{ \underset{0}{1}, \underset{1}{2}, \underset{2}{3}, \underset{3}{1}, \underset{4}{1} \}$$

B = 1

recur (A, B, 5, 3)	<table><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	0	0	0	1	2
0	0	0					
0	1	2					
recur (A, B, 4, ³ 2)	<table><tr><td>0</td><td>0</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	0	4	0	1	2
0	0	4					
0	1	2					
recur (A, B, 3, ² 1)	<table><tr><td>0</td><td>3</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	3	4	0	1	2
0	3	4					
0	1	2					
recur (A, B, 2, 1)	<table><tr><td>0</td><td>3</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	3	4	0	1	2
0	3	4					
0	1	2					
recur (A, B, 1, 1)	<table><tr><td>0</td><td>3</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	3	4	0	1	2
0	3	4					
0	1	2					
recur (A, B, 0, ¹ 0)	<table><tr><td>0</td><td>3</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	3	4	0	1	2
0	3	4					
0	1	2					

Ans =

0	3	4
---	---	---

```
int [] recur ( A, B, idx, cnt)
```

```
if (idx == A.length) return new int [cnt];
```

```
if (A[idx] == B) cnt = cnt + 1;
```

```
int [] res = recur (A, B, idx+1, cnt);
```

```
if (A[idx] == B)
```

```
    res[cnt-1] = idx;
```

```
return res;
```

TC: $O(N)$

SC: $O(N)$

0	1	2
1	3	4

Main

A[] = { 3 2 4 2 2 6 } B=2

idx = 0 1 2 3 4 5 6

cnt = 0 ~~1~~ 1 ~~2~~ ~~3~~ 3 3

Tower of Hanoi

There are n disks placed on tower A of different sizes

Goal → Move all disks from tower A to C using tower B if needed

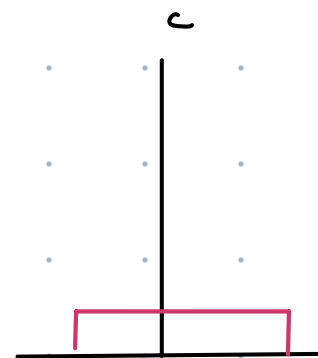
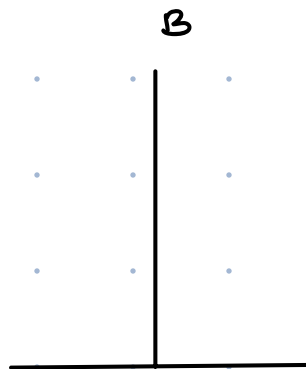
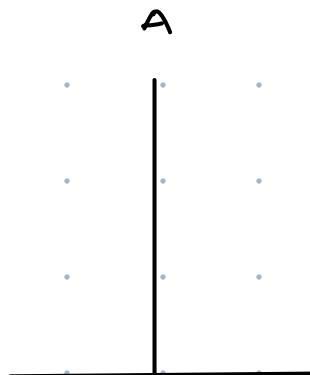
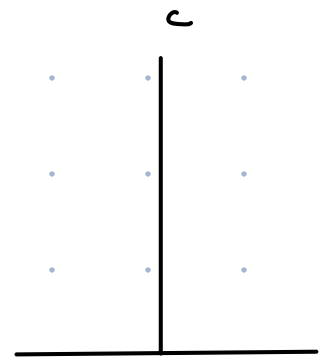
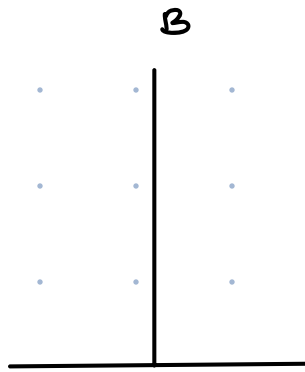
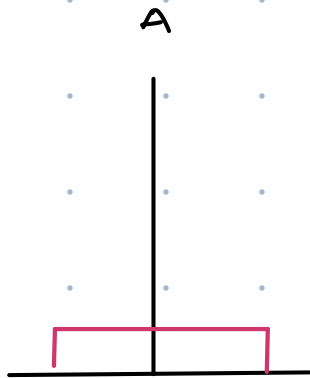
Constraint →

01. Only 1 disk can be moved at a time

02. Larger disk can't be placed at smaller disk at any step.

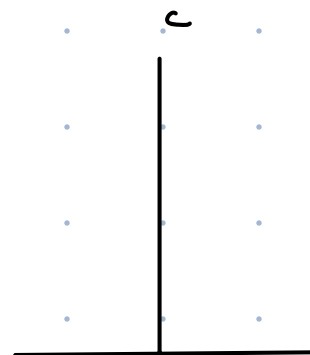
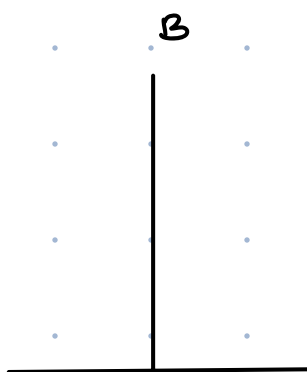
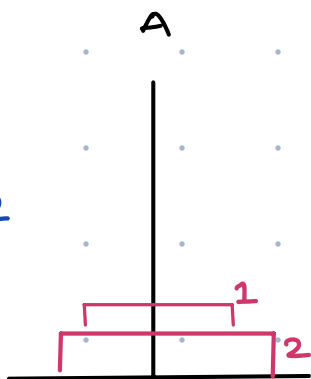
Print the movement of disks from A to C in minimum steps

$N=1$

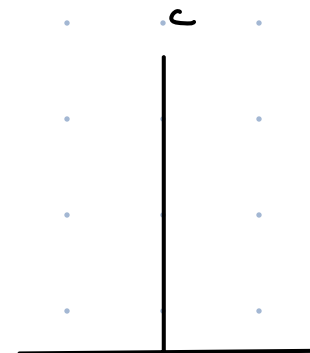
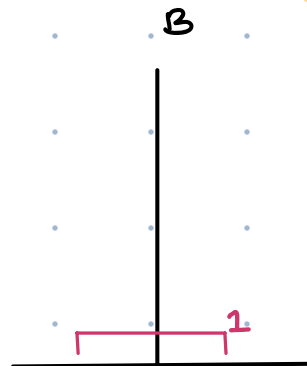
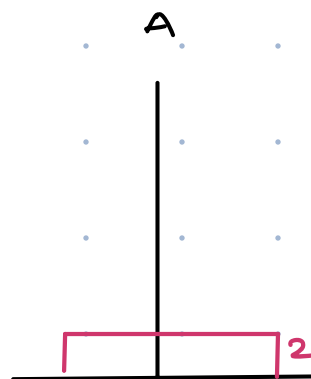


Output = 1 : A → C

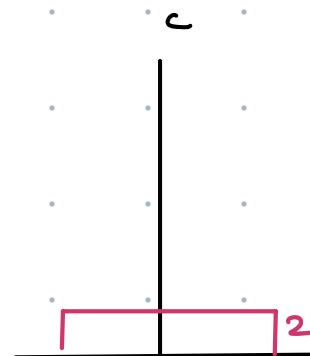
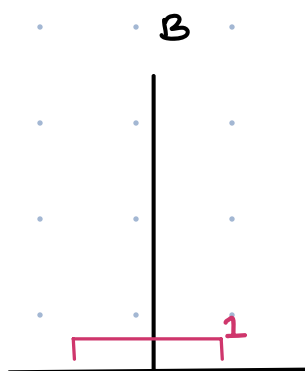
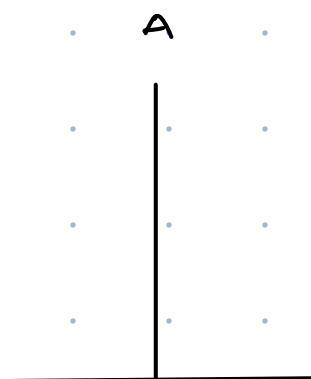
$N=2$



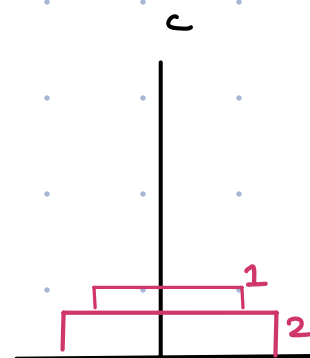
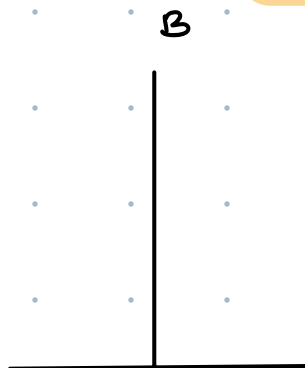
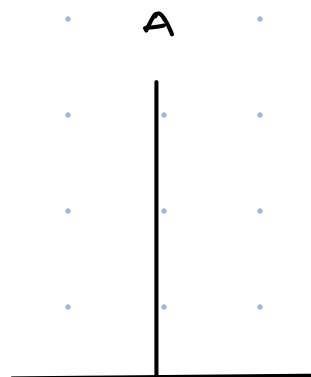
$1: A \rightarrow B$



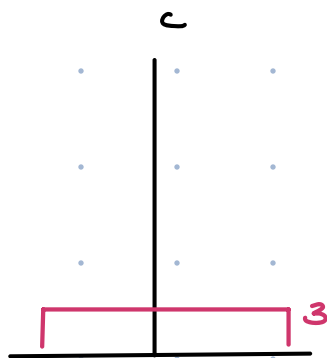
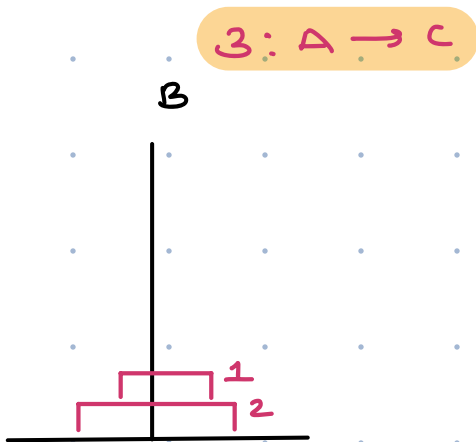
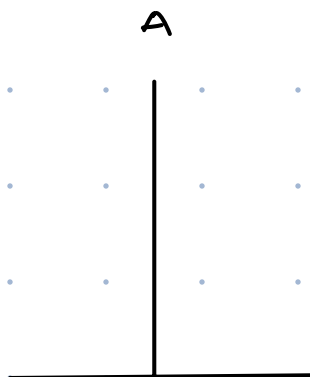
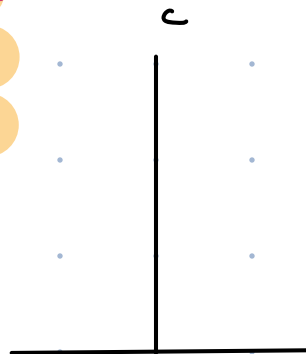
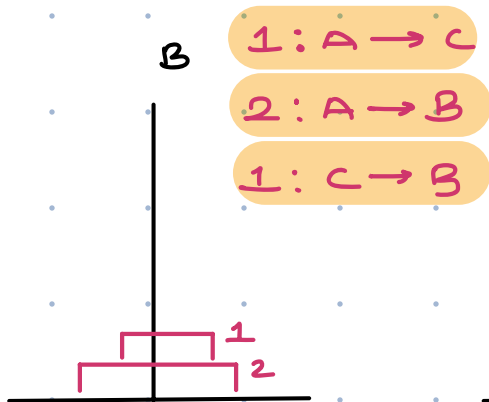
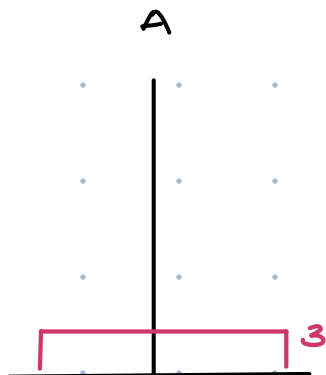
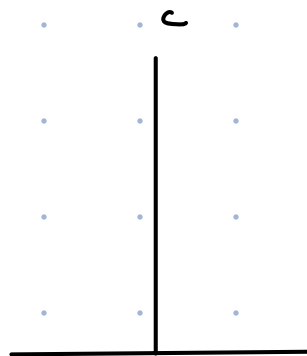
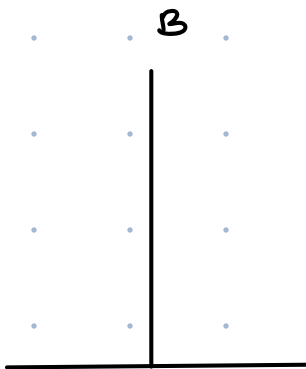
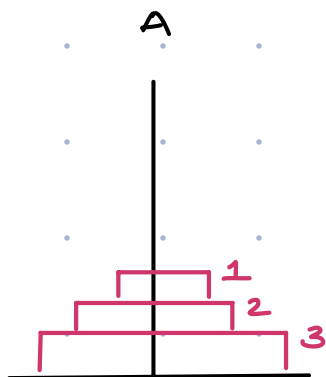
$2: A \rightarrow C$



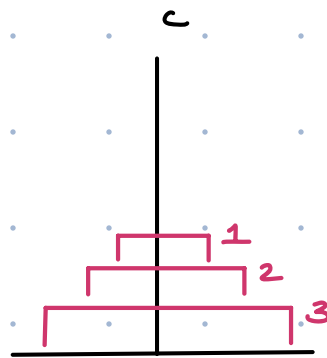
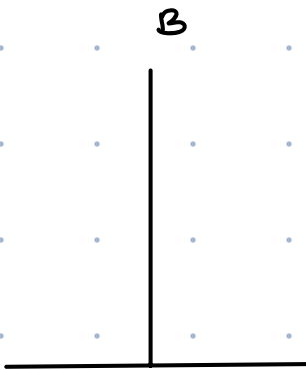
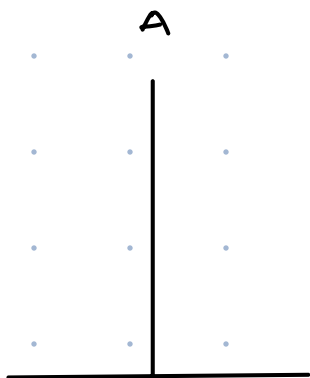
$1: B \rightarrow C$



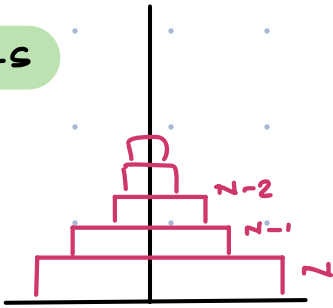
$N=3$



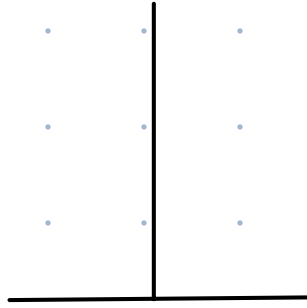
1: $B \rightarrow A$
2: $B \rightarrow C$
1: $A \rightarrow C$



A



B

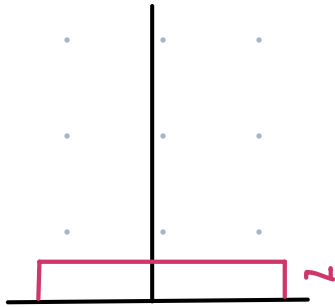


C

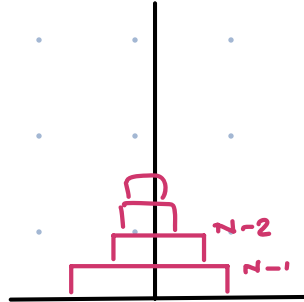


01 Move $(N-1)$ disks from src (A tower) to helper (B)

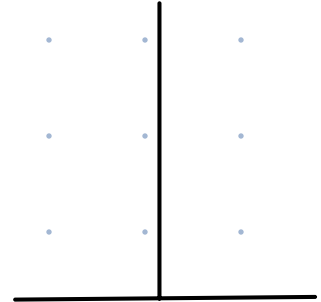
A



B

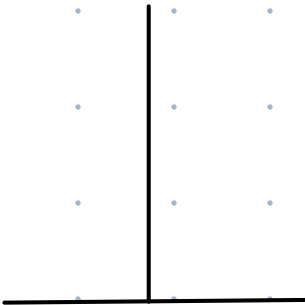


C

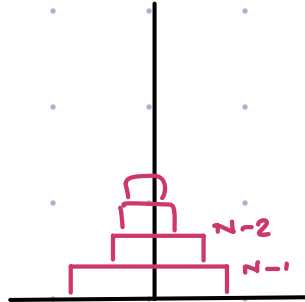


02 Move N disk from src to dest

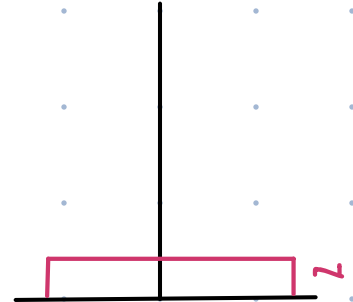
A



B

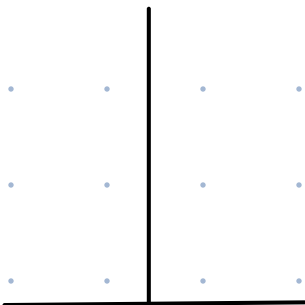


C

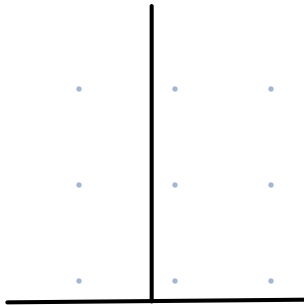


03. Move all $(N-1)$ disks from B tower to C tower

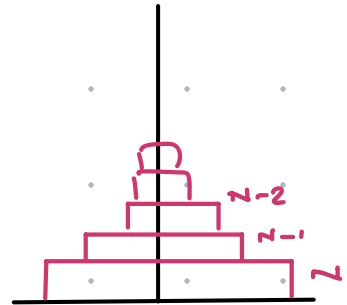
A



B



C



```

void TCH ( int n, A, B, C )
{
    if (n == 0) return;
    TCH (n-1, A, C, B);
    print (n: A → C);
    TCH (n-1, B, A, C);
}

```

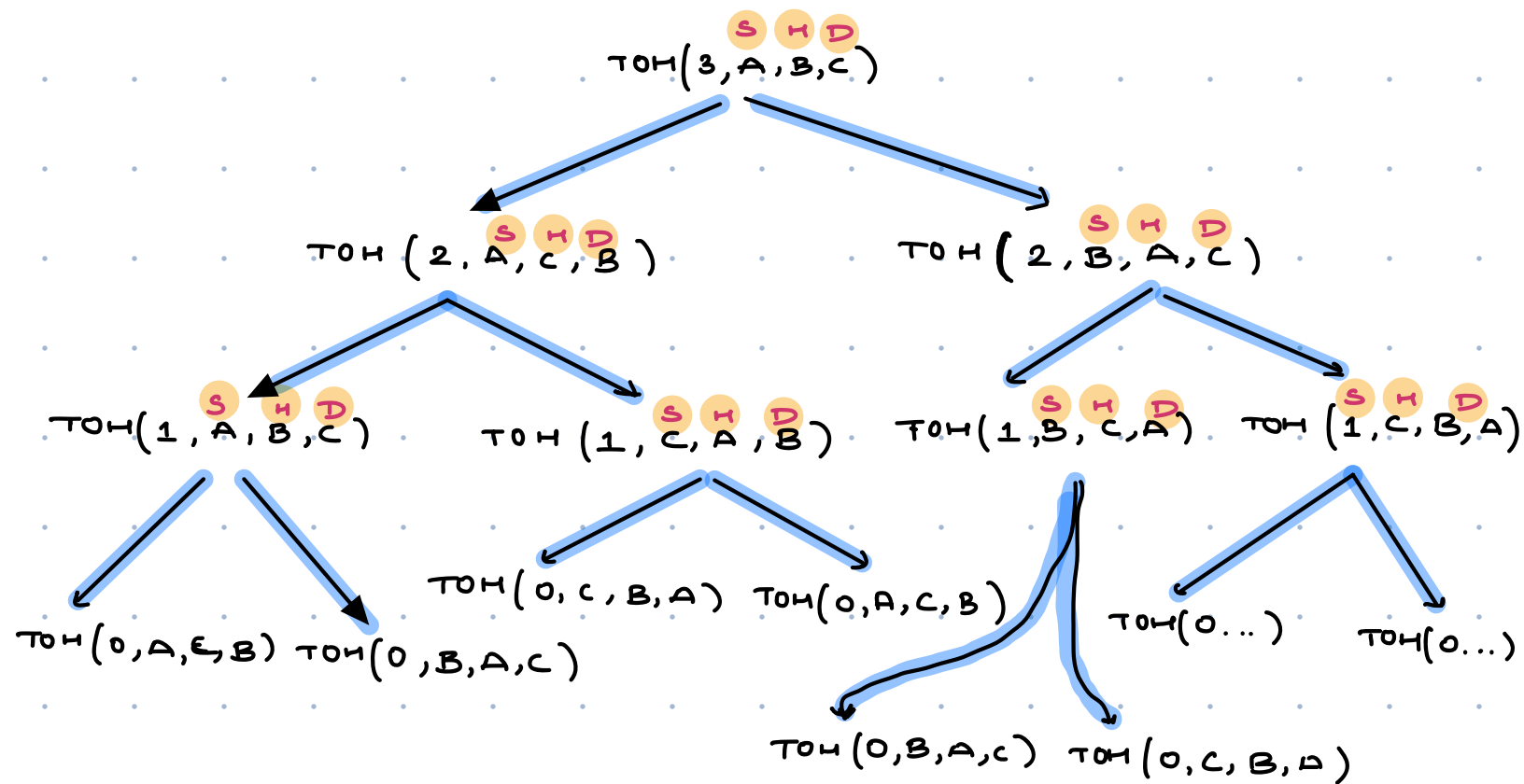
for tree

if (n == 0) return

left call = TCH (n-1, src, dest ← help)

print (n: src → dest)

right call = TCH (n-1, help ← src, dest)



1: A → C

1: A → C

2: A → B

2: A → B

1: C → B

1: C → B

3: A → C

3: A → C

1: B → A

1: B → A

2: B → C

2: B → C

1: C → A

1: A → C

TC: $O(2^n)$

SC: $O(n)$

